Complexity Analysis of Algorithm and Data Structure Choices

The Algorithms and Data Structures enhancement focused on improving the efficiency and scalability of the original animal rescue dashboard by selecting appropriate algorithms and data structures for common operations. The original application relied primarily on client-side processing, requiring large datasets to be transferred from MongoDB before filtering and aggregation occurred. Each enhancement below is evaluated strictly in terms of the complexity it changes. The trade-offs each choice introduces are discussed in the following section.

Compound index. Without an index, MongoDB performs a collection scan with time complexity approximately O(n), since every document must be examined. Adding a compound index for the frequently used animal_type, breed, sex_upon_outcome, and age_upon_outcome_in_weeks lets MongoDB use B-tree lookups instead, reducing matching-document search time to roughly O(log n) plus the size of the result set.

```
aac> db.animals.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  {
    v: 2,
    key: {
      animal_type: 1,
      breed: 1,
      sex_upon_outcome: 1,
      age_upon_outcome_in_weeks: 1
    },
    name: 'rescue_filter_index'
  }
]
```

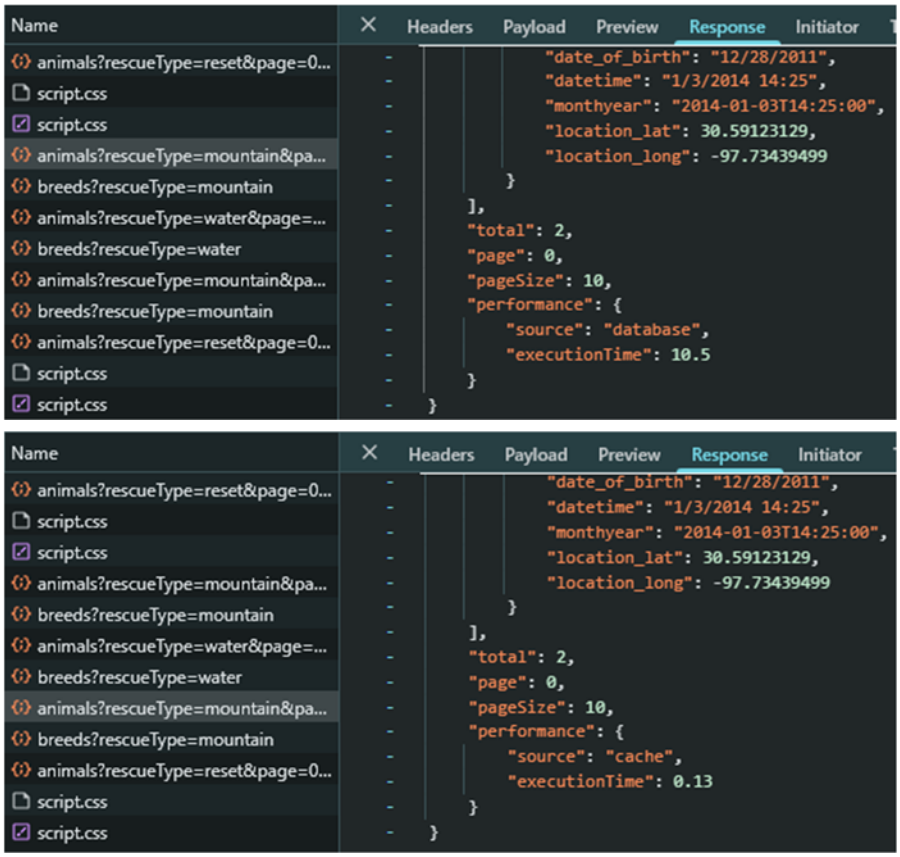
As shown in the figure on the side, the query plan uses an IXSCAN via rescue_filter_index (examining only 22 documents to return 22 matching results), confirming indexed lookup rather than a full collection scan.

```
executionStats: {
  executionSuccess: true,
  nReturned: 22,
  executionTimeMillis: 46,
  totalKeysExamined: 1005,
  totalDocsExamined: 22,
  executionStages: {
    isCached: false,
    stage: 'FETCH',
    nReturned: 22,
    executionTimeMillisEstimate: 19,
    works: 1005,
    advanced: 22,
    needTime: 982,
    needYield: 0,
    saveState: 1,
    restoreState: 1,
    isEOF: 1,
    docsExamined: 22,
    alreadyHasObj: 0,
    inputStage: {
      stage: 'IXSCAN',
      filter: {
        breed: {
          '$regex': '(labrador retriever
          '$options': 'i'
        }
      },
      nReturned: 22,
      executionTimeMillisEstimate: 9,
      works: 1005,
      advanced: 22,
      needTime: 982,
      needYield: 0,
      saveState: 1,
      restoreState: 1,
      isEOF: 1,
      keyPattern: {
        animal_type: 1,
        breed: 1,
        sex_upon_outcome: 1,
        age_upon_outcome_in_weeks: 1
      },
      indexName: 'rescue_filter_index',
      isMultiKey: false,
      multiKeyPaths: {
        animal_type: [],
        breed: [],
        sex_upon_outcome: [],
        age_upon_outcome_in_weeks: []
      },
```

Aggregation pipeline. Breed distribution was previously computed by retrieving full result sets and processing them client-side in pandas: an O(n) pass through data that had already been transferred over the network. Replacing this with a MongoDB aggregation pipeline ($match, $group, $sort, $project) moves the same O(n) filtering/grouping work into the database engine, which can use indexes during the $match stage and avoids transferring unfiltered data across the network at all.

Pagination. skip() combined with limit() bounds the size of each response to a constant number of records rather than the full collection, but skip() itself is O(k) in the offset k, since MongoDB must still advance through skipped documents. This makes it efficient for typical shallow-page access patterns but increasingly costly at large offsets (a limitation cursor-based pagination avoids by seeking directly from a bookmark rather than counting forward).

LRU cache. JavaScript's Map preserves insertion order while supporting O(1) get, set, and delete operations, which is what allows the cache to identify and evict the least-recently-used entry in constant time rather than requiring a separate tracking structure (e.g., a linked list) to maintain recency order.



API response before caching (source: database, 10.5ms) vs. after caching (source: cache, 0.13ms) for an identical repeated query.